API & REST API

What is an API?

API stands for **Application Programming Interface**.
It acts as a **bridge** that allows two different software systems to communicate and share data or functionality.

👉 **In simple terms:**
An API defines **how two software applications should talk to each other** what data they can exchange and how.

Examples of APIs

API Type	What It Does
Twitter API	Allows apps to fetch or post tweets programmatically.
Google Maps API	Enables websites or apps to display maps and get location data.
Stripe / PayPal API	Processes payments securely from your website or app.

What is the REST API?

REST stands for **Representational State Transfer**.
It is an **architecture style** for designing APIs that communicate over **HTTP** the same protocol used by web browsers.

Key Concept:

A **REST API** interacts with **resources** (like users, blogs, or products).
Each resource is represented as **JSON** (JavaScript Object Notation) data which is easy for both humans and computers to read.

Common HTTP Methods in REST APIs

HTTP Method	Purpose	Example
GET	Retrieve data	`/api/blogs` → Get all blogs
POST	Create new data	`/api/blogs` → Add a new blog
PUT / PATCH	Update existing data	`/api/blogs/:id` → Update a blog
DELETE	Remove data	`/api/blogs/:id` → Delete a blog

Example REST Endpoints for a Blog API

Endpoint	Description
`GET /api/blogs`	Fetch all blogs
`GET /api/blogs/:id`	Fetch one blog by ID
`POST /api/blogs`	Create a new blog
`PUT /api/blogs/:id`	Update a blog
`DELETE /api/blogs/:id`	Delete a blog

🗂 API Project Folder Structure (MVC Pattern)

To keep your backend code organized and scalable, developers use **MVC architecture Model, View, and Controller**. For APIs, it usually looks like this:

Component	Purpose
Server	Entry point starts the Express app and listens on a port
Routes	Defines API endpoints and maps them to controller functions

Controller	Contains the logic for handling each API request
Model	Defines how data is structured in the database (e.g., MongoDB Schema)

✅ **Benefit:** Keeps code **clean**, **modular**, and **easy to maintain**.

Testing APIs

We test APIs to **make sure they work correctly, reliably, and securely** before they are used by other applications or users.

It Ensures the API **does what it's supposed to do for** example, retrieving, creating, updating, or deleting data correctly.

- Example: When you send a `POST /api/blogs` request, a new blog should actually be created in the database.

  You can test Apis at the Browser or with applications Like Insomnia, Postman an Echo Api

Method	Description
Browser	Only works for simple `GET` requests. Example: `http://localhost:5000/api/blogs`
Postman / Insomnia / Echo API	Can test all methods (`GET`, `POST`, `PUT`, `DELETE`) and send JSON data easily.

Common HTTP Status Codes

When an API receives a request, it responds with a **status code** to tell the client (the user, app, or system) what happened.
These codes are **standardized by HTTP**, so they're universally understood across all web applications.

Each code is a **three-digit number**, and the first digit indicates the **category of the response**.

1xx — Informational Responses

These indicate that the request has been received and is being processed.

- **Example:** `100 Continue` → The server received the request, and the client should continue sending the rest of the data.

Why it matters: Rarely used in APIs but shows that the connection between client and server is working properly before full data transfer.

2xx — Success Responses

These indicate that the request was successfully received, understood, and processed.

Code	Meaning	When It's Used
200 OK	Request was successful.	Example: A `GET /api/blogs` request returns a list of blogs.
201 Created	A new resource was successfully created.	Example: A `POST /api/blogs` successfully creates a new blog post.
204 No Content	Request was successful, but there's no data to return.	Example: A `DELETE` request succeeds but doesn't need to return data.

✅ **Why we need them:** These confirm to the client that the action they performed (fetch, create, update, or delete) worked as intended.

🔁 3xx — Redirection Responses

These indicate that the client must take additional action to complete the request.

Code	Meaning	When It's Used
301 Moved Permanently	Resource has a new URL.	Used when API endpoints are moved permanently.
302 Found	Temporary redirect to a different URL.	Common during authentication (e.g., OAuth login redirects).

🔁 **Why it matters:** Helps maintain compatibility when resources move to new locations, avoiding broken links.

4xx — Client Error Responses

These indicate that the **client made a mistake** — for example, bad input or unauthorized access.

Code	Meaning	When It's Used
400 Bad Request	The request is invalid or missing data.	Example: Sending an incomplete JSON body.
401 Unauthorized	Authentication required or failed.	Example: Missing or invalid API key.
403 Forbidden	Users are authenticated but not allowed to access the resource.	Example: A normal user trying to delete admin data.
404 Not Found	The requested resource doesn't exist.	Example: Blog ID doesn't match any record.

409 Conflict	Conflict with existing data.	Example: Trying to create a record that already exists.

⚠️ **Why we need them:** These help users and developers quickly understand **what went wrong** in their request, making debugging much easier.

5xx — Server Error Responses

These indicate that the **server failed** to handle a valid request due to internal problems.

Code	Meaning	When It's Used
500 Internal Server Error	Generic server failure.	Example: Uncaught error in backend code.
502 Bad Gateway	Server received an invalid response from another service.	Common in microservices or proxy setups.
503 Service Unavailable	Server is temporarily overloaded or under maintenance.	Example: Too many API requests at once.

💥 **Why we need them:** They signal issues **inside the API**, helping developers identify and fix backend problems faster.

💻 Full Example: Blog REST API using Node.js + Express + MongoDB

Let's break down a real example step by step.

1. Controller (Logic Layer)

📁 `controller/blogController.js`

```
import Blog from "../models/blogModel.js";

// 1️⃣ Get all blogs
async function getAllBlogs(req, res) {
  try {
    const blogs = await Blog.find().sort({ createdAt: -1 });
    res.status(200).json(blogs);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// 2️⃣ Get one blog by ID
async function getOneBlog(req, res) {
  try {
    const blog = await Blog.findById(req.params.id);
    if (!blog) {
      return res.status(404).json({ message: "Blog Not Found!" });
    }
    res.status(200).json(blog);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

// 3️⃣ Create a new blog
async function postNewBlog(req, res) {
  try {
    const blog = await Blog.create(req.body);
    res.status(201).json(blog);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
}

// 4️⃣ Update a blog
async function updateBlog(req, res) {
  try {
    const blog = await Blog.findByIdAndUpdate(
```

```
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!blog) {
      return res.status(404).json({ message: "Blog not Found" });
    }
    res.status(200).json(blog);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
}

// 5️⃣ Delete a blog
async function deleteBlog(req, res) {
  try {
    const blog = await Blog.findByIdAndDelete(req.params.id);
    if (!blog) {
      return res.status(404).json({ message: "Blog not found" });
    }
    res.status(200).json({ message: "Blog Deleted Successfully!" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

export { getAllBlogs, getOneBlog, postNewBlog, updateBlog,
deleteBlog };
```

Explanation:

- `Blog.find()` → Fetches all blogs from MongoDB.
- `Blog.findById(id)` → Gets a single blog using its unique ID.
- `Blog.create(req.body)` → Adds a new blog using data from the request body.
- `Blog.findByIdAndUpdate()` → Edits an existing blog.
- `Blog.findByIdAndDelete()` → Deletes a blog by ID.

- `try...catch` → Handles success and errors cleanly using proper HTTP status codes.

2. Routes (Mapping Layer)

📁 **routes/blogRoutes.js**

```
import express from "express";
import {
  deleteBlog,
  getAllBlogs,
  getOneBlog,
  postNewBlog,
  updateBlog
} from "../controller/blogController.js";

const router = express.Router();

// Route definitions
router.get("/", getAllBlogs);
router.get("/:id", getOneBlog);
router.post("/", postNewBlog);
router.put("/:id", updateBlog);
router.delete("/:id", deleteBlog);

export default router;
```

Explanation:

- Each route defines a **URL path** and **method** (GET, POST, etc.).
- The route then calls a **controller function** to handle the logic.

Example:
When someone visits `/api/blogs`, the route calls `getAllBlogs()` in the controller.

3. Server Setup (Main Entry Point)

📁 **server.js or index.js**

```
import express from "express";
import mongoose from "mongoose";
import dotenv from "dotenv";
import blogRoutes from "./routes/blogRoutes.js";

dotenv.config();
const app = express();

// Middleware to parse JSON
app.use(express.json());

// Test route
app.get("/test", (req, res) => {
  res.send("API is running");
});

// Blog routes
app.use("/api/blogs", blogRoutes);

// 404 handler
app.use((req, res) => res.status(404).json({ message: "Route not
found" }));

// Database connection
const dbURI = process.env.MONGO_URI;

async function connectDB() {
  try {
    await mongoose.connect(dbURI);
    console.log("Connected to DB");

    app.listen(5000, () => {
      console.log("Server is running on http://localhost:5000");
    });
  } catch (error) {
```

```
    console.log("Mongo Failed to Connect");
  }
}

connectDB();
```

Explanation:

- `express.json()` → Parses incoming JSON requests.
- `/test` → Simple endpoint to verify the API is live.
- `mongoose.connect()` → Connects your app to MongoDB using the connection string from `.env`.
- The server listens on **port 5000**.

Summary

Concept	Description
API	Interface for software-to-software communication
REST API	Web API using HTTP methods for CRUD operations
MVC Pattern	Keeps API code organized (Model–View–Controller)
Postman / Insomnia	Tools to test all API endpoints
Express.js	Node.js framework to build APIs easily
Mongoose	Library to interact with MongoDB